

Asymptotic

A technical audit and development roadmap

Comparison with Wolfram Language, correctness risks, performance, API design, and reproducible improvement proposals

Repository	https://github.com/VladimirReshetnikov/Asymptotic
Pinned revision	75de8756175911cd8830704fd1a3406c1022f018
Commit timestamp	September 9, 2026, 19:16:26 UTC
Declared package	<code>AsymptoticInverse</code> 1.8.0; Wolfram Language 15.0 or later
Audit date	September 9, 2026
Prepared for	Vladimir Reshetnikov

Reading the evidence correctly. This report combines inspected source, official documentation, mathematical derivations, and executed independent Python checks. The 15 mathematical checks and five patch-transformation checks passed. No successful execution of the repository's Wolfram package or its regression suite was obtained during this audit. Wolfram regression cases and patch proposals are supplied for native validation, not presented as already validated fixes.

Contents

1	Executive assessment	3
1.1	Highest-priority findings	3
1.2	What this audit does not establish	4
2	Snapshot, method, and coverage	4
2.1	An immutable source boundary	4
2.2	Inspection approach	5
2.3	Execution record	5
2.4	Existing validation deserves credit, but not extrapolation	6
3	Architecture and mathematical contracts	6
3.1	A family of algebras, not one universal series type	6
3.2	Positive local coordinates	7
3.3	The ordinary precision algebra	7
3.4	Evidence classes that should remain distinct	7
4	The inverse calculus and its genuine added value	8
4.1	Complete-block Lagrange coefficients	8
4.2	Why remainder transport requires derivative information	8
4.3	Retained Gamma and Barnes cores	9
4.4	Finite sectors, oscillatory coefficients, and envelopes	9
5	Comparison with built-in Wolfram Language	10
5.1	The correct comparison baseline	10
5.2	How to compare concrete examples fairly	11
5.3	Where the package should explicitly defer	12
6	Detailed correctness and implementation findings	12
6.1	F01: optional native views can cause mandatory dense allocation	12
6.2	F02: real-power validation is bypassed inside nested observables	14
6.3	F03: an unseen logarithmic tail is treated too sharply	15
6.4	F04: ordinary forward constants lack a shared realness gate	16
6.5	F05: “commensurate” flat rates are more restricted than necessary	17
7	Other correctness boundaries: sound choices and residual risks	18
7.1	Composition must prove the outer domain on the actual inner germ	18
7.2	Do not confuse requested and available precision	18
7.3	A local interval certificate is not a global inverse theorem	18
7.4	Unknown or unsupported must remain distinct from false	19
8	Performance and scalability	19
8.1	Existing optimizations should not be proposed as new work	19
8.2	Multi-index growth versus output growth	19
8.3	One option currently represents several different resources	20
8.4	Hard-coded proof budgets and fallthrough can hide the real bottleneck	20
8.5	What the reported benchmarks do and do not say	21
8.6	Result and provenance size	21
9	API and user-experience design	21
9.1	Names and discoverability	21
9.2	Cutoff is an overloaded semantic parameter	22
9.3	The observable called Power changes meaning at one	22
9.4	Requested method versus effective method	23
9.5	A uniform result schema	23
9.6	Formatting, numerical evaluation, and Normal	23
10	Testing, packaging, documentation, and maintenance	24
10.1	A mathematical CI gate is the highest-leverage process improvement	24

10.2	Property-based and differential tests	24
10.3	Benchmark tests should include failure behavior	25
10.4	Distribution: keep one canonical implementation	25
10.5	License identifiers should match the actual text	25
11	A proposed development architecture	25
11.1	Separate mathematical primitives from dispatch policy	25
11.2	Make proof obligations inspectable	26
11.3	Typed scales and capability negotiation	26
11.4	Formal verification opportunities	26
12	Missing features and a prioritized roadmap	26
12.1	Stabilization before breadth	26
12.2	General multirate and nested transseries	27
12.3	Complex branches and Stokes behavior	27
12.4	Uniform parameter asymptotics	27
12.5	Integration, summation, and equation adapters	28
12.6	Success criteria for future development	28
13	Conclusion	28
A	Reproducing this audit and using the companion artifacts	28
A.1	Archive layout	28
A.2	Actual independent checks	29
A.3	Patch proposal generation	30
A.4	Native regression commands	30
A.5	Compiling the article	31
B	Regression and acceptance matrix	31
C	Coverage and source-navigation guide	32
D	An exact enclosure example independent of the package	33

1 Executive assessment

The repository is a substantial, useful asymptotic-calculus extension, but it should be positioned as a structured companion to Wolfram Language, not as a replacement for its asymptotic engine. Its distinctive contribution is the combination of sparse exact exponent arithmetic, complete logarithmic blocks, selected real inverse branches, retained inverse cores, explicit error transport, and reusable result objects. Much of the general special-function knowledge and ordinary series machinery already exists in Wolfram Language and is deliberately reused by the package. The strongest development strategy is to make the added contracts exceptionally reliable and interoperable.[12, 3, 8, 20, 23, 24]

The inspected snapshot has moved well beyond the original inverse-function problem. Its public interface covers ordinary forward and inverse expansions, logarithmic hierarchies, Fourier-valued coefficients, finite exponential sectors, Gamma and Barnes inverses, explicit series operations, numerical evidence, and selected rational-interval certificates. The generated standalone distribution combines 38 canonical kernel sources. The package guide explains many otherwise surprising restrictions, and its validation records carefully distinguish focused runs from the full suite. These are meaningful engineering strengths, not merely a collection of symbolic formulas.[12, 16, 14]

The principal concerns are concentrated at interfaces between these subsystems. A result’s optional native representation can allocate an enormous dense array during otherwise sparse computation. A real-power safeguard exists in one public operation but is absent from the recursive primitive used by another. An older native-series fallback infers an unknown logarithmic tail more aggressively than the newer importer. A restricted choice of exponential base rejects some genuinely commensurate rates. Constant-coefficient realness is not checked uniformly. These observations suggest that the next release should prioritize shared invariants and integration tests ahead of another large family of adapters.[3, 4, 11, 8, 9]

1.1 Highest-priority findings

Table 1 separates a demonstrated source property from a runtime observation. “Source-derived” means the relevant implementation was inspected and the stated control flow was traced; it does not mean the Wolfram example was executed successfully here. Severity measures the consequence if reached, not the frequency of occurrence.

Table 1: Priority findings and their evidence status.

ID	Priority	Finding	Evidence
F01	High	Eager, unbounded dense SeriesData conversion can defeat the sparse representation and MaxTerms .	Source-confirmed allocation; size derived exactly.
F02	High	A nested observable can bypass the real-branch check applied by direct SeriesPower .	Source-derived public control flow; native reproduction pending.
F03	High to investigate	The old native fallback can assign logarithmic degree zero to an unseen tail. An elliptic-integral example exposes the mathematical danger.	Unsafe inference identified; exact native shape and public reachability pending.
F04	Medium	Constant coefficients enter the ordinary forward jet without the realness checks used by other paths.	Source-derived contract inconsistency; native reproduction pending.

ID	Priority	Finding	Evidence
F05	Medium	Flat rates 2 and 3 are rejected although they have a common fundamental rate 1.	Explicit source restriction; mathematical fix supplied.
E01	High process priority	The latest milestone records focused tests, not a complete package run; the inspected CI workflow checks packaging rather than mathematics.	Repository’s own records and workflow.
E02	Medium	Resource, order, observable, and result-schema conventions vary across subsystems.	Documented API and inspected dispatch.
E03	Low, easy repair	Root license text says MIT No Attribution, while source and packet metadata say MIT.	Direct metadata discrepancy.

The near-term recommendation is a *contract-hardening release*: bound native-view construction, move branch obligations into shared operations, unify native-tail import, run complete native regression coverage, and publish a revision-pinned evidence manifest. The flat-rate generalization is a suitable small feature once its degree budget is also bounded. A more ambitious transseries framework should follow these repairs, not conceal them behind a growing dispatch tree.

1.2 What this audit does not establish

This report does not certify the entire package, prove every theorem in the repository article, establish a public speed advantage over Wolfram Language, or claim that built-in `AsymptoticSolve` fails on all logarithmic or irrational inverse problems. It also does not claim that the package’s interval certificate is invalid merely because its numerical seed is approximate: the inspected certificate arithmetic deliberately separates heuristic seed selection from exact rational verification.[7]

Conversely, inability to obtain a native run does not erase source-level evidence. An explicit unbounded `Table` has a determinable allocation length, and rejecting rates unless they are integer multiples of their minimum is a directly readable restriction. The appropriate response is to provide bounded reproducers, explain the mathematical expectation, and reserve execution claims for actual observations.

2 Snapshot, method, and coverage

2.1 An immutable source boundary

The audit is anchored to full commit

75de8756175911cd8830704fd1a3406c1022f018.

The commit message describes request-local logarithmic coefficient reuse and identical-envelope-operand reuse. It was newer than the initial snapshot encountered during inspection; the final report uses the newer revision. The principal core-file findings are unaffected by that change. Every repository reference in the bibliography points to the pinned revision rather than a moving `main` branch.[1, 3]

The packet manifest declares version 1.8.0 and Wolfram Language 15.0+. The root standalone file is reported by the Git tree as 574,627 bytes. This file is generated; the modular sources under `AsymptoticInverse/Kernel/` remain canonical. The two Git blob identities used by the patch emitter are recorded in Appendix A. A patch must not be silently applied to a different source revision.[17, 2, 16]

2.2 Inspection approach

The review followed data through the public entry points, coordinate normalization, sparse precision algebra, inversion, object construction, explicit operations, ordinary arithmetic, native special-function import, and certificate evaluation. Particular attention was paid to situations in which one component believes it has a stronger mathematical guarantee than another component actually supplied.

The following were examined in depth: the core kernel file; major parts of `SeriesOperations`, `SeriesArithmetic`, `InverseFunctionExpressions`, `InverseCertificates`, `NativeSpecialFunctions`, `ParameterizedSpecialFunctions`, `GammaInverse`, and `FlatSectors`; the current user guide; package metadata; the standalone-build workflow; validation records; and representative acceptance tests. The article’s entry file, section organization, abstract, and references were reviewed for intended mathematical scope. Other modules were inventoried or consulted selectively through their interfaces and callers. This is not a claim of exhaustive line-by-line review of every historical report, generated artifact, or test file.[3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 15]

Public Wolfram documentation was checked as current documentation on the audit date. Such documentation describes supported interfaces and intended behavior; it is not a substitute for a version-specific comparative execution. Native and package order parameters were therefore not treated as automatically equivalent.

2.3 Execution record

The execution boundary is unusually important for this audit. A Wolfram evaluator eventually returned the version string

15.0.0 for Linux x86 (64-bit) (May 6, 2026).

However, the subsequent request to download and load the pinned standalone package failed at the tool connection layer, and a follow-up probe also failed. No confirmed package load, package output, package source digest, native-tail probe, or full package test report was returned. The raw account is in `evidence/native-execution-status.txt`. A failed response cannot establish what, if anything, completed remotely.

Local independent checks did run under Python 3.13.5, SymPy 1.14.0, and mpmath 1.3.0. Fifteen mathematical and finite-model checks passed. Five additional tests of the patch emitter’s text transformations passed. The latter exercise excerpt fixtures and fail-closed anchors, not a complete parsed or executed Wolfram package. Logs and a machine-readable mathematical-results file are included.

Evidence category	Obtained here	Proper interpretation
Pinned source inspection	Yes, through repository tools	Grounds implementation and control-flow claims.
Official Wolfram documentation	Yes	Grounds documented native capabilities, not comparative success rates.
Independent mathematical checks	15 passed; zero failures/errors	Checks derivations and counterexample mathematics, not repository execution.

Evidence category	Obtained here	Proper interpretation
Patch-transformation fixtures	5 passed	Checks selected textual edits and drift rejection; not native semantics.
Native kernel version probe	One successful result	Establishes that probe’s kernel version only.
Repository Wolfram regression run	Not obtained	Supplied <code>.wlt/.wls</code> files remain to be run.
Repository-reported focused tests	Read, not independently rerun	Historical evidence with the scope stated by the repository.

2.4 Existing validation deserves credit, but not extrapolation

The newest validation record reports 135 passing tests in eight selected files, including ten new checks, with unchanged source hashes. It explicitly says that the full package suite was not run. The preceding generalized-series milestone reports 305 passing checks in 18 selected files and makes the same qualification. These are overlapping milestone records, not disjoint test populations to be added together.[\[14\]](#)

The selected acceptance tests include independent formulas, several numerical scales, retained remainder orders, cancellation, branch sides, and composition with vanishing derivatives. This is stronger than checking that outputs merely have the expected head. It nevertheless leaves a need for cross-dispatch adversarial cases: a property protected at a public wrapper can still fail when the same primitive is reached through an observable or a native fallback.[\[15, 4\]](#)

3 Architecture and mathematical contracts

3.1 A family of algebras, not one universal series type

The public head `GeneralizedSeries` wraps an association. Different constructors populate different keys: ordinary jets have power/logarithm blocks, factored expansions have an exact prefactor, transformed inverses retain coordinate series, Gamma/Barnes inverses retain Lambert cores, and flat or native special-function results may carry separate sectors or absolute envelopes. `Normal` extracts only the finite expression. The common head is useful, but common syntax does not by itself supply a common coefficient algebra.[\[3, 4, 12, 10, 8, 11\]](#)

A useful conceptual decomposition is

$$\begin{aligned} \text{result} = & \text{coordinate and branch} + \text{finite algebraic data} \\ & + \text{remainder contract} + \text{construction provenance.} \end{aligned}$$

Here the plus signs describe components, not numerical addition. Correct operations require compatibility in all four components. Equal-looking finite expressions need not have equal domains, derivative contracts, or omitted errors.

The inspected implementation already reflects much of this distinction. Ordinary arithmetic first tries a compatible ordered jet algebra, then may fall back to a composite absolute envelope. Flat sectors are handled separately. Differentiation requires a remainder-derivative contract. Native oscillations retain absolute bounds rather than being assumed bounded away from zero. These decisions should become explicit shared interfaces rather than remain distributed conventions.[\[4, 5, 8\]](#)

3.2 Positive local coordinates

For ordinary real approaches, the package uses a coordinate $u \rightarrow 0^+$:

$$u = x - x_0, \quad u = x_0 - x, \quad u = 1/x, \quad u = -1/x,$$

for a finite point from above, a finite point from below, positive infinity, and negative infinity, respectively. Source and target coordinates need not be the same. The inverse target coordinate also depends on the limiting target value and the sign of the leading coefficient. This is a sound basis for real powers and real logarithms.[3, 12]

A consequence is that an order such as u^3 may mean $(x - x_0)^3$ in one object and x^{-3} in another. A numerical value substituted into the displayed expression does not validate the selected asymptotic approach or prove that the approximation is accurate there.

3.3 The ordinary precision algebra

Write

$$J(u) = \sum_{\lambda \in S} u^\lambda P_\lambda(\log u) + \mathcal{O}(u^\rho M(u)^k), \quad M(u) = 1 + |\log u|, \quad (1)$$

where S is a finite ordered support and each P_λ is a polynomial. The core stores a precision-tracked jet essentially as a triple of rows, remainder exponent, and logarithmic degree. Exactness corresponds to infinite remainder exponent.[3]

For fixed real exponents and finite logarithmic degrees,

$$u^a M(u)^k = o(u^b M(u)^\ell) \quad (a > b),$$

regardless of k, ℓ . At equal powers, the larger logarithmic degree gives the weaker envelope. Thus an error comparison must not discard the logarithmic degree merely because the power is correct.

Addition must merge all equal-weight coefficients before determining which block survives. Multiplication must transport both input errors and their product. For finite expressions A, B and errors R, S , the basic envelope is

$$(A + \mathcal{O}(R))(B + \mathcal{O}(S)) = AB + \mathcal{O}(|A|S + |B|R + RS). \quad (2)$$

The newer native and composite arithmetic follows this principle. Identical operand extraction can be reused computationally without treating two error occurrences as cancelling.[8, 1]

3.4 Evidence classes that should remain distinct

A robust result protocol should distinguish the following claims. A *formal identity* says that algebraic coefficients cancel under substitution in the finite model. An *asymptotic contract* supplies an error class under hypotheses. A *numerical comparison* gives empirical agreement at selected points. A *certified enclosure* proves a pointwise containment with specified arithmetic and domain obligations. A *convergence theorem* is a different statement again.

The repository generally acknowledges these distinctions. In particular, `InverseResidual` states that its ordinary residual concerns the finite forward model, and `InverseNumericalCheck` is described as numerical evidence rather than an interval certificate. The certificate explicitly excludes unspecified functions represented only by `InputRemainder`. The proposed architecture should preserve these qualifications in machine-readable form and expose them prominently in the user interface.[3, 7, 12]

4 The inverse calculus and its genuine added value

4.1 Complete-block Lagrange coefficients

The ordinary inverse engine considers a normalized local model

$$v = au^p \left(1 + \sum_{i=1}^m u^{\delta_i} P_i(\log u) \right), \quad \delta_i > 0, \quad (3)$$

with a selected real branch. Set $z = (v/a)^{1/p} > 0$ and $L = \log z$. For an internal observable u^r , the coefficient construction implemented by `lagrangeCoefficient` has the form

$$u^r = z^r \sum_{\mathbf{k} \in \mathbb{N}_0^m} z^{\mathbf{k} \cdot \boldsymbol{\delta}} C_{\mathbf{k}}(L). \quad (4)$$

Here $C_0 = 1$. If $n = |\mathbf{k}| > 0$ and $w = \mathbf{k} \cdot \boldsymbol{\delta}$, then

$$C_{\mathbf{k}}(L) = \frac{(-1)^{n_r}}{p^n \prod_i k_i!} \prod_{j=1}^{n-1} (\mathcal{D} + r + w + pj) \prod_i P_i(L)^{k_i}, \quad \mathcal{D} = \frac{d}{dL}. \quad (5)$$

The empty operator product at $n = 1$ is the identity. This is the exact Euler-operator coefficient formula implemented in the core, not a numerical fit. The repository's mathematical article develops its analytic and asymptotic interpretations for stated classes. This audit checks representative coefficient consequences rather than claiming a new proof of all those extension theorems.[3, 13]

For $f(x) = x + x^2(1 + \log x)$, formula (5) gives

$$\begin{aligned} f^{-1}(y) &= y - y^2(1 + \log y) \\ &\quad + y^3(2 \log^2 y + 5 \log y + 3) + \dots \end{aligned}$$

The first correction is $-P$, with $P = 1 + L$. The second is $(\mathcal{D} + 4)P^2/2 = 2L^2 + 5L + 3$. Keeping the *entire* polynomial at a weight avoids a false ordering of its individual logarithmic terms.

For $f(x) = x + x^\beta$, $\beta > 1$, the same formula yields

$$\begin{aligned} f^{-1}(y) &= y - y^\beta + \beta y^{2\beta-1} \\ &\quad - \frac{\beta(3\beta-1)}{2} y^{3\beta-2} + \dots \end{aligned}$$

In particular, $\beta = \sqrt{2}$ does not fit one rational exponent lattice. The repository's explicit sparse weights and coefficient access are particularly natural for this case. These formulas were independently checked in the included Python tests.

Resonance matters even with rational gaps. For $f(x) = x + ax^2 + bx^3$, the inverse coefficient of y^3 is $2a^2 - b$. The multi-indices $(2, 0)$ and $(0, 1)$ have the same weight and must be combined. When $b = 2a^2$, the block disappears. A term goal counts the resulting nonzero blocks, not the number of multi-indices generated. The code's grouping and frontier logic are designed around this distinction.[3]

4.2 Why remainder transport requires derivative information

Suppose the finite model has leading derivative apu^{p-1} and an unknown additional term has value bound $\mathcal{O}(u^\rho M^k)$ with suitable corresponding derivative control. At the selected inverse branch, a perturbation argument gives

$$\Delta u = \mathcal{O}(u^{\rho-p+1} M^k).$$

For u^r , differentiation of the observable transports this to

$$\Delta(u^r) = \mathcal{O}(u^{\rho-p+r} M^k). \quad (6)$$

In the package's positive target coordinate, the corresponding power is $(\rho - p + r)/|p|$. At source infinity, the internal power is the negative of the requested source observable power. This explains the input-precision cap in the ordinary inverse constructor.[3]

Crucially, a bound on a function alone does not imply a bound on its derivative. For example, $R(u) = u^2 \sin(u^{-2})$ is $\mathcal{O}(u^2)$ but

$$R'(u) = 2u \sin(u^{-2}) - 2u^{-1} \cos(u^{-2})$$

is not $\mathcal{O}(u)$. The repository guide explicitly gives `InputRemainder -> {rho,k}` the stronger meaning of both a value bound and a matching first-derivative bound. This is *not* a missing assumption in the documented API. The improvement opportunity is to represent that obligation as a structured contract rather than a pair whose meaning must be remembered.[12]

4.3 Retained Gamma and Barnes cores

The Gamma inverse does not simply reverse a Taylor polynomial. For large target Y , write $\Lambda = \log Y$; for inverse `LogGamma`, write $\Lambda = Y$. The leading equation

$$X(\log X - 1) = \Lambda$$

has the exact Lambert solution

$$X = \frac{\Lambda}{W(\Lambda/e)}.$$

The inspected module computes corrections in powers of $1/X$ with whole polynomial coefficients in $q = 1/\log X$. Its first correction implies

$$x = X + \frac{1}{2} - \frac{\log(2\pi)}{2\log X} + \dots$$

This follows by dividing the logarithmic Gamma equation's leading correction by $\log X$. The code uses finite Stirling information and records a Poincaré contract, rather than asserting convergence of an infinite Stirling series.[10, 13]

For Barnes G the retained core is

$$X = \sqrt{\frac{4\Lambda}{W(4\Lambda/e^3)}}, \quad q = \frac{1}{\log X - 1},$$

with the Barnes argument reconstructed using its offset by one. Whole reciprocal-logarithmic coefficient blocks preserve a useful intermediate scale that would be obscured by immediately expanding every Lambert function into iterated logarithms. Affine source and target transformations and fixed powers have explicit restrictions in the parser. A claim of support for arbitrary products or compositions of inverse Gamma functions would go beyond that implementation.[10, 12]

4.4 Finite sectors, oscillatory coefficients, and envelopes

The flat engine treats a finite family of exponentially small contributions around an *exact* shifted monomial core. It refuses a merely truncated algebraic inverse as an exact zero sector. That refusal is mathematically appropriate: an omitted algebraic term can be larger than every retained exponential correction. Its current rate normalization is unnecessarily restrictive, as F05 explains, but the exact-core requirement itself is not a defect.[11]

Fourier-valued logarithmic coefficients and structured native special-function carriers broaden the supported examples without making the package a complete general transseries field. In particular, a sine or cosine coefficient is bounded but can have infinitely many zeros. It cannot be inverted as though it were a nonzero scalar coefficient. The newer native importer preserves separate error envelopes and checks the reality of relevant phases. This is a valuable bridge between native asymptotic formulas and explicit downstream error accounting.[12, 8]

5 Comparison with built-in Wolfram Language

5.1 The correct comparison baseline

A comparison limited to `InverseSeries` would be misleading. The relevant native baseline includes `Series`, `SeriesData`, `InverseSeries`, `Asymptotic`, `AsymptoticSolve`, asymptotic comparison predicates, and the integration, summation, differential-equation, and recurrence families. The native solver accepts real or complex centers, real-domain selection, conditions, and inferred nonstandard scales. A blanket claim that Wolfram Language has only rational-power Taylor reversion is false.[20, 21, 22, 23, 24]

`SeriesData` has a rational arithmetic-progression exponent structure, but this is a representation fact, not a global limitation on every expression returned by `Series` or `Asymptotic`. Logarithmic behavior and factors outside a series object must be considered separately. Likewise, built-in asymptotic order is not generally polynomial degree, while the package’s ordinary cutoff is a precise exclusive exponent bound. Comparisons must normalize these semantics before comparing output length or speed.[21, 20, 23, 24]

Table 3: Feature comparison: native capabilities versus the package’s incremental contribution.

Task	Native Wolfram Language	Repository and assessment
Ordinary local series	<code>Series</code> and <code>SeriesData</code> provide established local-series computation and arithmetic.	Broad overlap. Prefer native objects where their representation and contracts suffice.[20, 21]
Taylor/Puiseux inverse	<code>InverseSeries</code> handles suitable native series; <code>AsymptoticSolve</code> treats implicit equations.	Adds selected real source charts, complete-block cutoffs, explicit model access, and generalized weights.[22, 24, 3]
Irrational power weights	Native asymptotic functions can infer nonstandard scales; success is problem-dependent.	Exact sparse real weights are central rather than an incidental expression outside a rational grid.[23, 3]
Logarithmic inverse blocks	Native methods already address logarithmic and implicit asymptotic problems.	Explicit Euler coefficients, weight grouping, frontiers, and term-goal semantics are useful additions.[24, 3]
Real branches and conditions	<code>Reals</code> , <code>Direction</code> , <code>Assumptions</code> , and <code>GenerateConditions</code> are available in native solving.	Records source-side choices and provenance across supported inverse expressions; should interoperate rather than duplicate all solving.[24, 6]

Task	Native Wolfram Language	Repository and assessment
Lambert and rapid-growth inverses	Native <code>ProductLog</code> and asymptotic transformations provide relevant building blocks.	Dedicated retained-core Gamma/Barnes scales and operations provide a specialized workflow. No general native failure claim is made.[10, 12]
Special-function expansions	<code>Series/Asymptotic</code> already cover many special functions.	Mixes custom normalizations and defining sums with native import; added value is structured carriers and propagated errors.[20, 23, 8, 9]
Series arithmetic and composition	Already intrinsic to native series objects.	Extends operations to selected generalized scales and falls back to conservative composite envelopes. Not every scale supports every operation.[21, 4, 5]
Differentiation of remainders	Native series calculus differentiates its structured series representations.	Makes derivative-contract restrictions explicit for broader error descriptors.[21, 4]
Finite exponential sectors	Native exotic-scale asymptotics is relevant, but representation and order conventions differ.	Explicit finite sector depth and exact-core assumptions; currently not a general multirate transseries algebra.[23, 11]
Pointwise evidence	Native numerical solving and symbolic domain tools are available.	Distinguishes numerical comparison, finite-model residual, and a selected exact-rational certificate route.[3, 7, 29]
Integrals and sums	<code>AsymptoticIntegrate</code> and <code>AsymptoticSum</code> are direct built-ins.	No corresponding general public package engine appears in the documented interface. Integration should initially delegate or import structured results.[25, 26, 12]
Differential and recurrence equations	<code>AsymptoticDSolveValue</code> and <code>AsymptoticRSolveValue</code> address these domains.	Not a general replacement; a future adapter is more credible than broad reimplementations.[27, 28, 12]
Systems and multiple variables	Native implicit asymptotic solving supports systems and multiple independent variables.	The reviewed generalized inverse machinery is essentially one real source/target germ.[24, 12]

5.2 How to compare concrete examples fairly

For the regular inverse of $x + x^2$, a native baseline can use

```
InverseSeries[Series[x + x^2, {x, 0, 4}], y]
```

The package uses

```
AsymptoticInverse[x + x^2, {x, 0}, {y, 5}]
```

The common finite polynomial is $y - y^2 + 2y^3 - 5y^4$, derived directly from the inverse $(-1 + \sqrt{1 + 4y})/2$. This is a mathematical reference value, not a newly obtained native execution transcript. For this problem the package offers metadata and workflow advantages, not a new

inverse formula.[22, 12]

For $x + x^{\sqrt{2}}$ or $x + x^2 \log x$, the comparison must include native `AsymptoticSolve`, not only `InverseSeries`. A suitable task specification selects the solution tending to zero, a real target approach, and sufficient approximation depth. The included `CompareBuiltins.wls` records both sides without assuming that equal numeric order arguments request the same finite support. Any result comparison must first identify the selected branch and compare complete blocks.[24]

For Gamma ratios and special functions, compare a native factored output with the package's factored output before expanding carriers. The quotient of two growing factors can have a small correction bracket while its absolute expansion begins at a nonzero power. Conversely, two native special-function carriers may cancel and leave an ordinary amplitude scale. Benchmarks based only on string size or visible powers would mix different computational tasks.[12, 8]

The paired harness captures elapsed time, full input-form result, result head, version, and system ID, with a time and memory cap. It is a starting point, not a published benchmark: it was not successfully run during this audit, it does not prove semantic equivalence automatically, and a one-pass within-kernel ordering is inadequate for portable speed claims.

5.3 Where the package should explicitly defer

Use native `SeriesData` when a small rational lattice genuinely fits. Use native symbolic solving and domain reasoning as bounded proof oracles, not as silently trusted substitutes for a recorded branch condition. Preserve `ProductLog` when it is a useful exact core. Import native special-function asymptotics only through a checked conversion of the finite expression and its remainder structure. Leave general multivariate equation solving, asymptotic integration, summation, and differential equations to the corresponding native functions until a well-defined adapter contract is available.[21, 24, 25, 26, 27, 28, 8]

This division of labor makes the package more valuable. The user should gain a persistent, inspectable error-aware object around difficult asymptotic calculations, rather than a competing function name whose relationship to Wolfram's existing system is unclear.

6 Detailed correctness and implementation findings

6.1 F01: optional native views can cause mandatory dense allocation

Evidence status: Source-confirmed resource defect; allocation size checked independently. No large Wolfram allocation was attempted.

Source location: `AsymptoticInverse/Kernel/AsymptoticInverse.wl`, functions `makeSeriesData` and `makeInverseSeriesData`.[3]

Both conversion paths compute a common denominator for rational exponents and construct a full coefficient array extending to the remainder order. In simplified form, the relevant logic is

```
den = LCM @@ (Denominator /@ Append[exps, remPower]);
nmin = Min[exps] den;
nmax = remPower den;
coeffs = Table[0, {nmax - nmin}];
(* assign retained coefficients at their lattice positions *)
SeriesData[x, x0, coeffs, nmin, nmax, den]
```

The actual forward path handles empty support specially. That does not change the issue: the dense allocation is eager, has no size guard, and does not receive the construction's `MaxTerms` budget. The `SeriesData` property is built while the main result association is built, even when

the user never asks for that property.

Two distinct sources of waste

Consider the following family for an integer $N > 2$; the code uses the bounded representative $N = 200001$:

```
With[{n = 200001},
  AsymptoticExpansion[1 + x^(1/n) + x,
    {x, 0, 1}, "MaxTerms" -> 32]]
```

The retained generalized support is just $\{0, 1/N\}$; the omitted x supplies remainder exponent 1. The original conversion allocates N zeros, although only two adjacent coefficients are retained. A native series stores its remainder order separately from its coefficient-list length, so this trailing zero padding is unnecessary.

Now add a second widely separated fractional term:

```
With[{n = 200001},
  AsymptoticExpansion[1 + x^(1/n) + x^(1 - 1/n) + x,
    {x, 0, 1}, "MaxTerms" -> 32]]
```

There are only three retained sparse blocks, but a genuine rational-grid native view requires slots from 0 through $N - 1$. Eliminating trailing padding is no longer enough: there is a large interior gap. For $N = 10^9$, the inspected formula calls for one billion coefficient entries. The included checks calculate this count with integer arithmetic; they do not allocate the array.

This is a performance and robustness bug, not merely an inconvenient native representation. The optional view can prevent completion of a valid sparse calculation. Its input encoding is small, its sparse support is small, and `MaxTerms` does not protect the allocation. Exact memory consumption depends on Wolfram's representation and must not be guessed from an assumed bytes-per-entry constant.

Why inspecting the final object can miss the problem

`SeriesData` may canonicalize away trailing zero coefficients after construction. A short returned coefficient list does not prove that the temporary `Table` was short. A correct regression therefore needs either allocation instrumentation or a preallocation guard; checking only final `ByteCount` or `Length` can miss the first example entirely.

The companion regression suite uses a bounded $N = 200001$ for the wide-interior-gap case. A separate short-coefficient-list test checks the intended native view, but is explicitly not evidence that the baseline avoided a large temporary allocation.

Recommended fix

The best interface makes native conversion lazy and explicitly fallible. A sparse expansion should remain usable even when its native view is uneconomical. A native-view method can return

```
Missing["DenseSeriesDataLimit",
  <|"RequiredCoefficients" -> count, "Limit" -> cap|>]
```

without invalidating the `GeneralizedSeries` itself.

Before allocation, compute the number of *retained* lattice slots,

$$n_{\text{slots}} = \begin{cases} 0, & S = \emptyset, \\ 1 + d \max S - n_{\min}, & S \neq \emptyset, \end{cases}$$

where d is the common denominator and $n_{\min} = d \min S$. Keep the remainder index $n_{\max}$ separate. Apply an absolute slot cap and, optionally, a density test comparing slots with retained support size. The same helper should serve forward and inverse views.

The supplied emitter implements an interim version: no unnecessary trailing padding and a preallocation cap of 100,000 retained lattice slots, configurable when generating the proposal. It intentionally does not pretend to implement lazy properties or a general memory-budget framework. The sparse constructor survives a refused view. Tests must cover empty support, negative starting exponents, rational denominators, inverse coefficients, and invariance of the finite expression and remainder.

6.2 F02: real-power validation is bypassed inside nested observables

Evidence status: Source-derived public control-flow defect; native reproduction pending.

Source location: `seriesPower` and `seriesJetApply` in `SeriesOperations.wl`; shared `fwdPower` in the core kernel.[\[4, 3\]](#)

The explicit series-power wrapper already protects against a positive noninteger power of a pure finite remainder. It rejects an input whose finite support is empty and whose error is nonzero, because a magnitude bound does not establish the real radical branch. This is good behavior.

However, `seriesJetApply`, used recursively by `SeriesObservable`, sends a nested power directly to `fwdPower`. That primitive contains the following earlier return, before a known leading coefficient is checked:

```
If[T === {},
  If[less[0, rr],
    Return[{{}}, If[P === Infinity, Infinity, rr P],
      Ceiling[rr D]], Module],
  fail["UnknownLeadingTerm", ...]]
```

For a positive fractional `rr`, it transports the magnitude exponent without establishing that the underlying quantity is nonnegative. The special top-level shortcut for an observable exactly equal to a bare power does not protect powers deeper in an expression tree.

Minimal public-path reproducer

```
s = AsymptoticExpansion[-x, {x, 0, 1}];
SeriesPower[s, 1/2]
SeriesObservable[s, 1 + Sqrt[z], z]
```

The first constructor retains no finite term: the result represents a zero finite expression with an $\mathcal{O}(x)$ remainder on $x \rightarrow 0^+$. The direct power is guarded. The recursive observable instead reaches the early return above, and source tracing predicts a finite expression 1 with an $\mathcal{O}(x^{1/2})$ remainder rather than a branch failure.

The intended original source is $-x$, whose principal square root is not real for $x > 0$. More fundamentally, the abstract information $\mathcal{O}(x)$ alone cannot distinguish positive, negative, and sign-changing realizations. It therefore cannot certify a real square root. This finding is a

real-branch contract violation; it is not an assertion that $\sqrt{-x}$ fails a complex magnitude estimate of order $x^{1/2}$.

Recommended fix and compatibility boundary

Place the uncertain-real-power check in the shared primitive before the pure-remainder return. The supplied proposal rejects a noninteger power when support is empty and precision is finite. It leaves an exact zero distinct from an unknown zero-sized approximation and preserves the existing known-positive-leading-term path.

A richer future contract could carry a proved nonnegativity property and permit selected pure-remainder powers. That would require the proof to survive all transformations; a generic value bound or a discarded source coefficient is insufficient. Do not repair this by replaying arbitrary source expressions only in one wrapper, because it would reintroduce route-dependent semantics.

The supplied tests require both direct and nested uncertain square roots to fail, while a known-positive leading term remains accepted. Broader tests should place the same fractional power beneath addition, multiplication, exponential, reciprocal, and conditional-observable nodes. The expected invariant is independent of the syntactic route.

6.3 F03: an unseen logarithmic tail is treated too sharply

Evidence status: Unsafe internal inference identified; mathematical counterexample established independently. The exact native series shape and public constructor reachability have not been reproduced.

Source location: Core `fwdSeries`, especially its empty-candidate branch; compare `specialNativeTree` in `NativeSpecialFunctions.wl`.[\[3, 8\]](#)

The older ordinary fallback asks native `Series` for one order, converts its visible coefficients, and asks again at one higher order. It tries to infer the first omitted coefficient's logarithmic degree from the second result. When no such coefficient is visible, it leaves `kdeg = 0` and raises the remainder exponent to the second series object's endpoint:

```
If[cand != {},
  kdeg = polyDegree[(* first visible omitted coefficient *), ell];
  rho = (* its exponent *),
  rho = sd2[[5]]/sd2[[6]]
]
```

The missing premise is that the unknown coefficient at that endpoint has logarithmic degree zero. An absent visible block says nothing of the kind.

This is especially notable because the newer native importer explicitly addresses the same issue. It says that native `SeriesData` does not encode the logarithmic degree of its unknown tail, and conservatively loses half a lattice step when forming an absolute power envelope. The old and new paths therefore embody different precision policies.[\[8\]](#)

A mathematically decisive test family

Let $K_{\text{par}}(m)$ denote the complete elliptic integral with *parameter* m , as in Wolfram's `EllipticK[m]`. DLMF uses modulus notation in its corresponding expansion. Set $m = 1 - u^2$ and $L = \log(4/u)$. The complementary-modulus expansion gives

$$K_{\text{par}}(1 - u^2) = L + \frac{u^2}{4}(L - 1) + \frac{9u^4}{64}\left(L - \frac{7}{6}\right) + \mathcal{O}(u^6 L), \quad u \rightarrow 0^+. \quad (7)$$

The coefficients follow from DLMF 19.12.1 and 19.12.3 after translating between parameter and modulus.[32, 30]

Subtract the first two displayed blocks and call the residual $R(u)$. Then

$$\frac{R(u)}{u^4} = \frac{9}{64} \log(1/u) + \mathcal{O}(1) \longrightarrow +\infty, \quad \frac{R(u)}{u^4 \log(1/u)} \longrightarrow \frac{9}{64}.$$

Therefore $R \notin \mathcal{O}(u^4)$, although $R \in \mathcal{O}(u^4 M)$ and $R \in \mathcal{O}(u^{7/2})$. There is no cubic block. If the first native probe ends at the cubic order and the second probe contains no visible cubic coefficient but an endpoint at order four, the old empty-candidate branch produces the invalid pair $\{4, 0\}$.

The conditional clause matters. Native **Series** could return a different expression shape on a particular version; the old fallback might reject it; and the public special-function constructor can route this example through the newer, safer importer. This audit does *not* assert that the public `AsymptoticExpansion[EllipticK[1-u^2], ...]` was observed to return the wrong bound. The included probe records native shapes, the old private conversion, and the public result separately.

u	R/u^4	$R/(u^4 \log(1/u))$	$R/u^{7/2}$
10^{-2}	0.678534	0.147342	0.0678534
10^{-4}	1.326089	0.143978	0.0132609
10^{-8}	2.621293	0.142302	0.000262129
10^{-12}	3.916497	0.141743	0.00000391650

These numbers were computed independently with `mpmath` at high precision. They support the derivation, but the divergence proof comes from (7), not a finite sample.

Recommended fix

Unify native import through one contract-aware conversion. When a tail is known only to have a fixed finite logarithmic polynomial at a native lattice endpoint ρ , return a slightly weaker exponent $\rho - \varepsilon$, or retain an explicitly unknown logarithmic degree until more information is obtained. The implication

$$u^\rho M(u)^k = \mathcal{O}(u^{\rho-\varepsilon}) \quad (\varepsilon > 0, k \text{ fixed})$$

is rigorous. It is not a theorem about arbitrary unrecognized coefficient scales, so the importer must still state the admitted tail class.

The optional patch proposal changes only the empty-candidate branch to a half-lattice-step loss. It is a narrow mitigation, not a complete unification and not a proof that every native fallback is sound. A finite additional peek must never be upgraded into proof that the next unknown coefficient has no logarithm. Regression coverage should include lacunary special-function series, mixed logarithmic powers, Puiseux denominators, and identical inputs reaching different dispatch routes.

6.4 F04: ordinary forward constants lack a shared realness gate

Evidence status: Source-derived contract inconsistency; native reproduction pending.

Source location: Core `pConst` and the `FreeQ[e,u]` arm of `fwd`; compare `rowsToModel`, regular arithmetic operands, and parameterized special-function checks.[3, 5, 9]

A coefficient independent of the local variable enters `pConst`, which checks that it is polynomial in the logarithm placeholder but does not apply the realness proof used by several other subsystems. The ordinary forward route can therefore carry symbolic coefficients whose nonreality follows from the supplied assumptions.

A focused regression candidate is

```
AsymptoticExpansion[Log[-a] + x, {x, 0, 2},
  Assumptions -> a > 0]
```

For real positive a , the principal logarithm has imaginary part π . Merely being independent of x does not make the coefficient real. The package can manipulate the resulting formal expression, but that is different from meeting its advertised real-expansion contract. The related parameterized-special-function path explicitly verifies real coefficients, making the policy discrepancy concrete.

The proper design decision is to choose and publish a coefficient domain. In a strict real mode, a provably nonreal coefficient must be rejected. An unproved coefficient may require conditions or an explicit formal mode. A future complex mode would need branch and sector contracts rather than silently inheriting real-coordinate conclusions.

A centralized coefficient validator should receive the assumptions and the declared coefficient variables, including the reality of the logarithm placeholder. This must be tested for positive and negative parameters, symbolic real parameters, undefined functions, and exact transcendental constants. The companion regression records the desired rejection above; the patch emitter does not implement this broader policy change.

6.5 F05: “commensurate” flat rates are more restricted than necessary

Evidence status: Source-confirmed support limitation; exact normalization proposal supplied.

Source location: `flatModel` in `FlatSectors.wl`.[\[11\]](#)

The flat engine requires a common positive phase power and chooses the smallest rate as its base. It then requires every rate divided by that minimum to be an integer. This admits rates $\{1, 2, 3\}$ and $\{2, 4, 6\}$, but rejects $\{2, 3\}$. The latter are genuinely commensurate: both are integer multiples of 1. The implemented class is “integer multiples of the smallest *present* rate,” a stricter class than general finite commensurability.

For example,

```
AsymptoticFlatInverse[
  x + Exp[-2/x] + Exp[-3/x], {x, 0}, {y, 3}]
```

reaches the explicit `IncommensurateFlatRates` restriction. A valid formal sector marker is $q = \exp(-1/y)$, with input contributions at degrees 2 and 3. Through degree 3, the inverse is

$$x = y - e^{-2/y} - e^{-3/y} + \text{terms in sectors of degree at least four.}$$

The polynomial factors accompanying higher sectors require their own bounds; the expression above is not shorthand for a coefficient-free $\mathcal{O}(q^4)$ assertion.

A rational greatest-common-divisor construction

Choose a positive reference rate c_0 and prove that every ratio $r_i = c_i/c_0$ is a positive rational. Write the ratios in lowest terms and set

$$g = \frac{\gcd_i \text{num}(r_i)}{\text{lcm}_i \text{den}(r_i)}, \quad c_* = c_0 g.$$

Then c_i/c_* are positive integers. For $\{2, 3\}$, the reference 2 gives ratios $\{1, 3/2\}$, $g = 1/2$, and fundamental rate $c_* = 1$.

This can also admit a common exact algebraic factor when ratio simplification proves rationality. It must not use floating-point near-integer tests: rates 1 and $\sqrt{2}$ are not commensurate. A small denominator can produce a very large normalized degree, so the normalization must be followed by a degree/resource check before the existing dense polynomial-sector operations are entered.

The supplied proposal implements exact rational-ratio normalization, rechecks integer degrees, and caps the maximum degree using the existing limit. A later sparse sector polynomial would allow large unused gaps more naturally. General incommensurate rates require multiple markers or a finitely generated ordered rate monoid, not rounding to a convenient common base.

7 Other correctness boundaries: sound choices and residual risks

7.1 Composition must prove the outer domain on the actual inner germ

The ordinary composition routine checks that the inner local coordinate has a positive monomial leading block and transports the outer power/logarithm remainder. It also constructs an output domain by substituting the finite inner expression into the outer domain. That substitution is not, by itself, a proof that a strict domain condition holds for every realization of an uncertain inner germ.[4]

This is a high-value audit target, not a reported reproduced bug. Away from boundaries, a positive leading separation can prove eventual containment. At a boundary, a discarded error may determine which side is entered. A future domain-transport helper should consume the whole precision-tracked jet, much as the observable-condition logic already does, and return proved, disproved, or unresolved status. Tests should combine narrow conditional domains, translations, and inner remainders that touch a boundary.

7.2 Do not confuse requested and available precision

Truncating an existing object cannot recover information beyond its input remainder. Increasing an operation's requested cutoff must either replay a trusted source/refinement recipe or preserve the old precision cap. The inspected precision triples, observable checks, and refinement mechanisms show substantial attention to this distinction. Regression tests should make it a universal law across ordinary, transformed, flat, reciprocal-logarithmic, and composite objects.[4, 12]

A useful property test is monotonicity: weakening input information must not strengthen output guarantees without an independent proof. Another is path consistency: applying an observable directly should agree, up to the declared error, with an equivalent composition route. F02 is precisely the kind of cross-route failure such tests can expose.

7.3 A local interval certificate is not a global inverse theorem

The certificate evaluator uses exact rational endpoints and directed dyadic rounding. Its supported expression grammar includes rational arithmetic and selected exponential/logarithmic operations with explicit tail bounds. It checks the selected source side, retained source-domain condition over the entire closed verification interval, continuity through supported evaluation, and derivative separation from zero. Approximate arithmetic is used to propose a seed, not to settle the final containment test.[7]

If c lies in a verification interval, $|f'(x)| \geq \mu > 0$ there, and $|f(c) - y| \leq \epsilon$, then an established root satisfies $|x_* - c| \leq \epsilon/\mu$. The existence premise is essential. The inspected implementation

supplies it through residual-bracket containment or exact endpoint signs, then tightens the enclosure with the signed derivative interval. This is a meaningful certificate for the stored explicit function on that interval.

It does not certify all unknown functions sharing a declared `InputRemainder`; it does not imply a global branch theorem; and its public kind restriction does not admit every generalized inverse family. Those are documented scope limits, not evidence of a broken certificate. Extension to new functions should add independently reviewed enclosures rather than merely permit new heads in a switch.

There is, however, a concrete UX inconsistency: `InverseCertificate` advertises an option default `"Interval" -> Automatic`, but its entry validation immediately requires an ordered pair of exact rational endpoints. Thus the default is effectively a missing mandatory argument, not automatic interval discovery. Either make the interval an explicit required argument or implement a bounded discovery phase that still returns an unproved failure when verification cannot be completed.[7]

7.4 Unknown or unsupported must remain distinct from false

Several helpers use bounded symbolic reasoning and dispatch fallbacks. This is necessary for a symbolic package, but an undecided comparison or a timeout must not be interpreted as a proof of the opposite statement. Exact exponent ordering, eventual-domain tests, positivity, nonzero leading coefficients, and exact termination each need a three-way outcome with a separate resource-limit state.[3, 6, 9]

The appropriate goal is not to force every input into an answer. It is to return a result whose declared contract is justified, or a diagnostic explaining the unsatisfied obligation. A refusal on an unsupported scale is preferable to a sharper-looking expression with an unproved remainder.

8 Performance and scalability

8.1 Existing optimizations should not be proposed as new work

The inspected implementation is not uniformly naive. Sparse jet multiplication exploits sorted weights to find admissible product columns before multiplying coefficients, and checks the product budget before allocating the retained product list. Integer powers use binary powering. Multi-index boundary neighbors are deduplicated as they are generated. Grouped Lagrange and Newton methods are already present. The newest change introduces a request-local logarithmic coefficient builder, including cached zero coefficients, and narrowly reuses identical composite-operand data.[3, 1]

A review recommending simply “add memoization,” “use sparse multiplication,” or “implement Newton inversion” would miss this work. The remaining problem is to make optimization decisions and budgets consistent across the entire request, including native views, proof attempts, normalization, and formatting.

8.2 Multi-index growth versus output growth

For positive gaps δ_i and a weight cutoff H , ordinary enumeration considers

$$\left\{ \mathbf{k} \in \mathbb{N}_0^m : \sum_i k_i \delta_i < H \right\}.$$

With fixed positive gaps, its leading volume estimate grows like $H^m/(m! \prod_i \delta_i)$ as the cutoff grows; boundary effects and resonance affect the exact count. Many multi-indices can contribute to a single output weight. Thus the number of nonzero output blocks can be much smaller than the amount of coefficient work.

For commensurate gaps, grouped convolution may avoid repeated work over resonant indices. For noncommensurate gaps, support ordering and exact exponent comparison become important costs. A term goal further complicates matters: zeros can force exploration past many candidate weights. The repository already implements grouping, incremental state, and bounded termination checks; future profiling should distinguish their costs instead of attributing everything to the final term count.[\[3, 1\]](#)

The useful counters are candidate indices, distinct weights, zero blocks, polynomial degree, coefficient expression size, exponent-comparison count, and proof-oracle time. Report these alongside elapsed time. A large speedup caused by silently returning a weaker remainder is not a valid optimization.

8.3 One option currently represents several different resources

MaxTerms is used as a bound on sparse product pairs, multi-indices, expression leaves, requested orders, and other local work units. These units are not interchangeable. A single enormous exact coefficient can be expensive with one retained term; a rational exponent with a large denominator can trigger F01; and symbolic simplification or exact integer factorization can dominate without increasing a term count. The core's logarithm canonicalization and repeated symbolic reasoning therefore deserve profiling even after sparse multiplication has been optimized.[\[3, 9, 8, 10\]](#)

Replace the informal global meaning of **MaxTerms** with a request budget whose dimensions are explicit. A proposed shape is

```
"ResourceBudget" -> <|
  "SupportEntries" -> 20000,
  "ProductPairs" -> 200000,
  "CoefficientLeaves" -> 100000,
  "DenseViewEntries" -> 100000,
  "ProofSeconds" -> 5,
  "TotalSeconds" -> 60
|>
```

These numbers are illustrative policy defaults, not measured recommendations. Retain **MaxTerms** as a compatibility shorthand mapping to documented defaults. Preflight allocations where possible, check a total deadline between stages, and return the resource that was exhausted together with the work already justified.

8.4 Hard-coded proof budgets and fallback can hide the real bottleneck

Native-series probes and domain checks have different fixed timeouts. The parameterized-special-function wrapper can spend up to its local timeout, convert a failed or inapplicable attempt into **\$Failed**, and let the ordinary dispatcher try another route. A costly input may therefore pay several budgets sequentially and eventually receive an unhelpful unsupported-input message.[\[9, 8, 6\]](#)

Use explicit outcomes such as **NotApplicable**, **UnsupportedScale**, **UnprovedCondition**, and **ResourceLimit**. The dispatcher may fall through on the first; it should preserve the reason and work record for the others. A diagnostic trace should show which adapters were considered, the effective method, the chosen scale, proof obligations, and stage timings. This would make both

performance engineering and user support much easier.

Request-local caching is a good default because assumptions, branch selections, coefficient variables, and source state matter. Global memoization without those components risks cross-request contamination. The current local-cache direction should be extended through a common context object, not replaced by unbounded global caches.[1, 6]

8.5 What the reported benchmarks do and do not say

The latest repository record compares immutable commit 07a9781212beb2eeb9ff16aa625b50ac27974078 with the newer construction changes in fresh Wolfram 15.0.1 kernels. It reports one warm-up and three measured runs per fixture, and exact agreement of finite expressions and remainders in eight fixtures. Selected medians are reproduced below as *repository-reported* values, not measurements from this audit.[14]

Fixture	Before (s)	After (s)
Generalized logarithmic inverse, six blocks	8.09769	8.08571
Cancelled logarithmic resonance, five blocks	6.14373	6.09454
Identical Gamma inverse composite, addition	0.00170	0.00093
Identical Gamma inverse composite, multiplication	0.00116	0.00085
Exact scalar added to Gamma inverse composite	0.00153	0.00130

The record itself says the public logarithmic times are essentially unchanged because bounded exact-composition checks dominate. It does not claim a general logarithmic speedup. The millisecond arithmetic fixtures should not be used to infer whole-workload performance. The earlier internal coordinate/Fourier improvements are likewise scoped microbenchmarks, not a comparison with built-in Wolfram asymptotic functions.[14]

A useful future benchmark suite should have cold-load, warm public-call, and internal-kernel categories; randomize execution order; run isolated kernels; retain every sample; record hardware and source hashes; and compare equivalent branch/error targets. Peak memory and failed cases belong in the report. Performance thresholds should tolerate platform noise but catch asymptotic regressions such as dense lattice allocation.

8.6 Result and provenance size

Derived series retain recipes referring to their operands, and inverse-expression provenance can retain inverse series. This is valuable for refinement but can produce large serialized expression trees after repeated composition and arithmetic. The current audit did not measure pathological growth, so this is a profiling target rather than a demonstrated memory leak.[4, 6]

Separate the mathematical value from an optional provenance DAG with stable node identifiers. Provide compact inspection and a serialization mode that can omit or externalize caches. Avoid retaining duplicate complete operand histories when a shared immutable node would suffice. Cache removal must not remove information required to justify a result or reconstruct its branch.

9 API and user-experience design

9.1 Names and discoverability

The repository is named `Asymptotic`, the package/context is `AsymptoticInverse`, and its primary forward entry point is `AsymptoticExpansion`. The name reflects the project's origin

but now understates its scope. This is not a reason to introduce a second abrupt rename. A clearer top-level description, reference index, and stable context would be preferable.[12, 17]

Version 1.8.0 replaces `PowerLogSeries` with `GeneralizedSeries` without an exported compatibility alias. The guide explicitly warns that patterns and saved inputs must be updated. This is a real compatibility cost, even though it is documented. Add an import/migration function and a schema version for saved objects. Preserve old input migration separately from the runtime public head so that compatibility does not require every internal function to recognize multiple unvalidated representations.[12, 14]

9.2 Cutoff is an overloaded semantic parameter

The guide carefully explains that ordinary powers, correction brackets, inverse-log coordinates, retained cores, marker degrees, sector degrees, Fourier weights, and defining-sum weights have different order conventions. Some cutoffs are exclusive; marker and sector depths are inclusive. A multi-carrier term goal can return more than the requested total number of blocks because it applies per carrier. This is documented complexity, not missing documentation.[12]

The API can still make these distinctions more visible. Introduce typed requests such as

```
TruncationSpec["Power", "Coordinate" -> w,
  "Below" -> h]
TruncationSpec["RelativePower", "Below" -> h]
TruncationSpec["SectorDegree", "Through" -> n]
TruncationSpec["NonzeroBlocks", "Count" -> n,
  "Scope" -> "PerCarrier"]
```

These are proposed interfaces, not existing symbols. Existing positional forms should translate to a normalized request object. A result should expose both `RequestedTruncation` and `AchievedTruncation`, the actual coordinate, the first excluded weight when known, and whether a term goal was reached or exact termination occurred.

A lightweight `ExplainRequest` operation could state, before substantial computation, “retain powers below h in $1/X$, with each reciprocal-logarithmic coefficient kept whole.” That is more useful than a generic list of options, especially when automatic dispatch changes the scale family.

9.3 The observable called `Power` changes meaning at one

At a finite source endpoint, `"Power" -> r` for $r \neq 1$ returns a displacement power $(x - x_0)^r$, while $r = 1$ returns the original source variable including its translation. In contrast, applying `SeriesPower[s,r]` to a result is naturally understood as a power of the whole represented expression. The distinction is explained in the guide but remains easy to miss in reusable code.[12, 3, 4]

A proposed replacement is an explicit observable:

```
"Observable" -> Function[x, x^2]
"Observable" -> Function[x, (x - x0)^2]
```

with an optional, separately named local-coordinate observable. Implement both through the same shared error-transport machinery used by `SeriesObservable`. Keep the old `Power` option as a documented compatibility translation. This both clarifies the API and reduces opportunities for direct-versus-nested validation differences such as F02.

9.4 Requested method versus effective method

The ordinary inverse defaults to "Lagrange" and also admits "Newton" and "GroupedLagrange". The Gamma/Barnes inverse constructor accepts these ordinary method names as supported options but computes its family-specific ordered corrections through the same coefficient construction path. A user selecting Newton should be able to determine whether Newton was actually used.[\[3, 10\]](#)

Expose `RequestedMethod`, `EffectiveMethod`, and `MethodSelectionReason`. An eventual `Method` $\rightarrow$ `Automatic` can choose among existing ordinary algorithms using gap structure, support size, and coefficient complexity, but should first be evaluated against reproducible public benchmarks. A supplied explicit method should either be honored, reported as translated, or rejected with an actionable explanation.

9.5 A uniform result schema

The present association-based representation is flexible, but operations repeatedly branch on `Kind`, `Scale`, `SeriesRepresentation`, and optional coordinate keys. The arithmetic object's preliminary structural check requires only a few keys. This is not a validation of the full mathematical invariants. Hand-constructed or old saved associations can therefore reach deeper code with mismatched metadata.[\[5, 4\]](#)

The public contract should discourage direct construction and provide a validator. A proposed common schema includes:

```
<|"SchemaVersion" -> 2,
  "Germ" -> <|"Variable" -> x, "Point" -> x0,
    "Direction" -> side, "Domain" -> domain|>,
  "Scale" -> scaleDescriptor,
  "FinitePart" -> sparseData,
  "ErrorContract" -> errorDescriptor,
  "Evidence" -> evidenceDescriptor,
  "Recipe" -> recipeReference|>
```

Scale-specific data can remain in dedicated payloads. Common operations should depend on capabilities such as exact coefficient addition, ordered support, coordinate differentiation, absolute envelopes, and branch proofs, rather than arbitrary string tests scattered across modules.

The validator should check positive local coordinate meaning, coefficient-domain declarations, support ordering and uniqueness, remainder/retained-support consistency, and which derivative orders are justified. Validation should not attempt to prove every analytic theorem afresh; it should verify that the required evidence or trusted-constructor tag is present and well formed.

9.6 Formatting, numerical evaluation, and Normal

The pretty representation deliberately hides the `GeneralizedSeries` head and displays a familiar finite expression plus an O term. Copying the formatted object preserves the underlying object; `Normal` explicitly drops the remainder. These are thoughtful usability choices. However, an attractive O display can obscure the distinction between an asymptotic class, a numerical error bound, and a certified enclosure.[\[12\]](#)

A compact expandable summary should show the active coordinate, branch, remainder interpretation, and evidence status. The numerical application `s[value]` evaluates the finite expression; it should not be mistaken for a guaranteed approximation with a prescribed number of correct digits. A separately named checked-evaluation operation could verify the recorded domain and

return the value together with any available numerical or certified error evidence. Failure to have a pointwise bound should be explicit, not silently replaced by the magnitude of the formal remainder scale.

For conditions, standardize diagnostics around the expression being proved, assumptions used, source/target role, and eventual versus pointwise scope. `FunctionDomain` or other native domain tools may help, but a computed domain and a proof of eventual containment along a selected germ remain different tasks.[29, 6]

10 Testing, packaging, documentation, and maintenance

10.1 A mathematical CI gate is the highest-leverage process improvement

The inspected `.github/workflows/standalone.yml` runs the standalone freshness check and Python builder tests. Its path filters concern kernel sources, the standalone artifact, builder files, and the workflow itself; a change only to Wolfram test files does not trigger that workflow. These are facts about the inspected workflow, not a claim that no other external test process exists. Combined with the explicit absence of a full-suite run at the latest milestones, they leave a release-assurance gap.[18, 14]

Keep the fast packaging check. Add a native smoke gate covering every public family and all newly fixed invariants, then a complete native suite for releases and scheduled runs. Native-kernel availability, licensing, and runner isolation must be arranged within the project's environment. A test infrastructure failure should be distinguishable from a mathematical test failure and from an intentionally skipped family.

Every report should retain commit hash, source-tree digest, native version, platform, test selection, pass/fail/skip counts, elapsed time, and abnormal kernel termination. A report generated by one kernel version must not silently serve as proof of a later version's compatibility. Treat generated standalone parity as an independent test dimension: passing the modular source is not enough if packaging changed contexts or load order.

10.2 Property-based and differential tests

The highest-value additions are cross-path tests rather than more snapshots of long expressions. Generate small admissible sparse models and check complete coefficient blocks by independent substitution. Compare Lagrange, grouped Lagrange, and Newton on the same domain and cutoff. Check resonant cancellation, sparse gaps, negative endpoints, exact termination, and unknown leading terms.

For operations, test information monotonicity, branch consistency, and equivalent syntax. In particular, every guarded operation should also be reached through `SeriesObservable`, `SeriesNormalize`, composition, and arithmetic upvalues. Test native import through both direct special-function and fallback routes where possible. Native comparison should use independent coefficients or normalized residual bounds, not require identical printed expressions.

A useful metamorphic test begins with a known invertible function f and an exact affine change of source or target. Compare the transformed inverse with the transformation of the original inverse, including remainder scale and branch side. Another changes an input remainder from a stronger to a weaker bound and checks that the output does not spuriously improve. Such tests directly target the boundaries exposed by this audit.

10.3 Benchmark tests should include failure behavior

Add safe allocation sentinels for large rational denominators, large normalized flat degrees, and small requested support with expensive coefficients. A resource failure should preserve a useful diagnostic and should not leave package-global state changed. Test aborted computations followed by an ordinary successful call. The direct large- N allocation example must be capped before running it in CI; deliberately exhausting a runner is not a useful regression test.

Warm and cold loading, reloads, **Needs**, context pollution, saved objects, and numerical precision should also have explicit tests. The repository already performs several isolated loading checks; keep them and include their selection in release manifests.[\[16, 14\]](#)

10.4 Distribution: keep one canonical implementation

The generated standalone file is a practical distribution mechanism. The builder checks dependencies and source freshness, and the development notes explain why `URLDownload` followed by local `Get` replaced direct compressed HTTP loading after observed Wolfram 15.0.1 problems. That workaround should not be removed merely for aesthetic reasons.[\[16\]](#)

The getting-started guide defaults to a moving `main` URL while also documenting pinned commits. For reproducible research, make a versioned release or immutable commit the first recommended path, publish a digest, and keep “latest development” explicit. Package loading executes code, so a checked immutable artifact is a stronger default than an unversioned network dependency.

The `paclet` manifest declares only a kernel extension. The repository does have a substantive Markdown/HTML guide and a mathematical article; it would be inaccurate to call it undocumented. Native `paclet` reference pages, guide pages, examples, and F1 discoverability remain useful additions. Generate these from a stable source and test their examples rather than maintaining multiple drifting copies.[\[17, 12, 16\]](#)

10.5 License identifiers should match the actual text

The root `LICENSE` is titled “MIT No Attribution License.” The main source header declares `SPDX-License-Identifier: MIT`, and `PacletInfo.wl` declares `"License" -> "MIT"`. These identifiers do not describe the same license text. This is a directly observed metadata inconsistency, not an opinion about legal enforceability.[\[19, 3, 17\]](#)

Choose the intended license consistently. For the current no-attribution text, align identifiers with MIT-0; alternatively adopt the intended standard MIT text consistently. Include the license choice in generated distributions and release metadata. This repair is small, but it removes avoidable ambiguity for downstream packaging and automated scanners.

11 A proposed development architecture

11.1 Separate mathematical primitives from dispatch policy

The most important refactor is not a wholesale rewrite. Extract a small trusted-by-design kernel of operations that all public paths use: coefficient-domain validation; exact support normalization; remainder comparison and transport; real-power/logarithm admission; positive local-coordinate composition; and proof-status handling. F02 demonstrates why a wrapper-only check is insufficient. F03 demonstrates why two independent native-tail import policies should not persist.

Keep family adapters responsible for their own normalization identities and specialized coefficient

formulas. Require each adapter to state its coordinate, coefficient domain, tail theorem or imported error source, branch obligations, and capabilities. The dispatcher should select an applicable adapter and report that choice, rather than accumulate increasingly broad exception handlers.

11.2 Make proof obligations inspectable

Introduce an evidence record with categories such as `ExactAlgebra`, `AsymptoticTheorem`, `NativeSeriesImport`, `UserDeclaredContract`, `NumericalEvidence`, and `RationalIntervalCertificate`. Each should state its hypotheses and scope. A user's derivative-bound declaration must remain distinguishable from a bound derived by the implementation.

An operation can combine evidence without pretending to strengthen it. For example, multiplying two Poincaré expansions yields a transported asymptotic contract, not automatically a numerical certificate. Normalizing an exact identity may preserve all evidence; truncation weakens information; refinement may add information only by replaying a justified recipe. This record would make results easier to review, serialize, and eventually formalize.

11.3 Typed scales and capability negotiation

A scale descriptor should expose ordered support where available, coefficient operations, a realization map to ordinary expressions, and an absolute-envelope operation. Some algebras support differentiation under specified contracts; some support only arithmetic or selected observables. Operations should ask for those capabilities before falling back to an envelope.

This permits a principled distinction between a coefficient algebra and an asymptotic order. A Fourier coefficient algebra has bounded modes but not general nonvanishing; a reciprocal-logarithmic polynomial algebra has its own coefficient ordering; a finite flat-sector algebra has an exponential degree plus amplitude information. Treating all of them as ordinary power-log rows with extra flags invites accidental assumptions.

11.4 Formal verification opportunities

A realistic formalization target is the small algebraic/error kernel, not the whole Wolfram evaluator. Start with exact sparse support merge, truncated convolution, the product-envelope rule (2), and rational interval arithmetic. Next formalize the Euler-operator coefficient identity and the finite-order inverse perturbation theorem with explicit derivative hypotheses.

Native `Series`, `Simplify`, and domain solvers can then act as external producers of candidates or certificates rather than opaque proofs. Checking a polynomial identity or a rational interval derivation is much smaller than verifying every symbolic algorithm that discovered it. The repository's explicit coefficient access and certificate structure make this a plausible development direction, but no formal verification of the current implementation is claimed here.

12 Missing features and a prioritized roadmap

12.1 Stabilization before breadth

Release A: contract hardening. Repair F01 and F02, reproduce and reconcile F03, settle the forward coefficient-domain policy in F04, and align license metadata. Add direct-versus-nested operation regressions and a native smoke gate. Acceptance means no unbounded native-view

allocation, no unproved real radical through a recursive route, and no native-tail upgrade without a stated contract. Publish the complete native test selection and outcomes.

Release B: predictable requests and results. Introduce normalized truncation requests, requested/effective method metadata, result schema versioning, a validator, a checked evaluation path, and explicit evidence summaries. Preserve compatibility translations. Acceptance means that a user can inspect the actual coordinate and remainder meaning without reading a family-specific implementation file.

Release C: measured scalability. Extend the existing request-local computation state, add multi-dimensional resource budgets, instrument proof-oracle time, make native views lazy, and benchmark method selection. Acceptance means equivalent finite coefficients, branch domains, and remainder guarantees before and after optimization, plus bounded peak-memory behavior for sparse high-denominator inputs.

Release D: focused mathematical extensions. Generalize rationally commensurate flat rates first, then consider multiple independent exponential rates, broader exact-core observables, and more consistent algebra on reciprocal-logarithmic and composite results. Each new family must bring a tail theorem, branch/domain contract, cross-operation tests, and failure cases. Expanding a list of recognized function heads is not sufficient.

12.2 General multirate and nested transseries

The current finite sector model is not a general treatment of arbitrary nested exponential/logarithmic scales. A multi-rate extension should use exact rate vectors and a supported ordering, explicitly handle resonant rates, and keep the algebraic zero sector accurate enough relative to the requested exponential sectors. General nesting requires a valuation tower and clear closure conditions for composition and inversion.

This is a substantive research and engineering project. A useful intermediate deliverable is a finite generated scale algebra with explicit capabilities and conservative errors, not a claim of universal transseries support. The commensurate-rate repair in F05 is a small, testable step toward that architecture.

12.3 Complex branches and Stokes behavior

Complex asymptotics require sectors or approach curves, branch cuts, and potentially Stokes-dependent exponentially small contributions. Extending a real sign test to **Complexes** is not enough. A future complex mode should be a separate declared contract with sector-aware powers, logarithms, native imports, and continuation rules.

The near-term package should continue to refuse unsupported complex behavior explicitly. The F04 real-coefficient issue should be resolved as a domain-policy decision, not used to advertise accidental formal complex manipulation as complex asymptotic support.

12.4 Uniform parameter asymptotics

Parameter assumptions are not the same as uniform error bounds. For fixed $a > 0$, $e^{-a/u}$ is smaller than every power as $u \rightarrow 0^+$, but along $a = u$ it equals e^{-1} . A fixed-parameter flat estimate does not hold uniformly through $a \rightarrow 0$. Likewise, a leading coefficient approaching zero can change the dominant balance and invalidate an inverse chart.

A future interface should distinguish fixed parameters, compact parameter domains bounded away from degeneracy, and joint limits. Uniform expansions near coalescing balances may require different scales. This is an improvement opportunity and a warning against overinterpreting

ordinary assumptions, not a reproduced defect in the documented fixed-parameter contract.

12.5 Integration, summation, and equation adapters

The package’s explicit remainder objects could become useful inputs and outputs for native asymptotic integration, summation, differential-equation, and recurrence solvers. Start with an importer/exporter that states what information is preserved and what is lost. Do not claim that a generic O descriptor can always be integrated termwise over an unbounded or parameter-dependent domain.

Prioritize adapters that reuse established native functionality and add verifiable metadata. A new standalone asymptotic differential-equation engine would be a much larger undertaking with little immediate benefit compared with stabilizing the current inverse calculus.[25, 26, 27, 28]

12.6 Success criteria for future development

A successful extension should improve at least one of four measurable dimensions: admitted mathematically justified input class, strength and clarity of the error/branch guarantee, reproducible resource behavior, or composability of results. A shorter printed expression alone is not enough. The project will be strongest when its output tells the user not only an asymptotic formula, but precisely which germ it represents, which operations remain valid, and why its remainder can be trusted.

13 Conclusion

The repository has a coherent mathematical center and a substantial engineering investment. Its explicit inverse coefficients, complete-block treatment, retained cores, branch-aware transformations, and conservative error operations can support workflows that are awkward to organize using finite expressions alone. Built-in Wolfram functionality remains an essential foundation and comparison baseline.[3, 12, 4, 10, 8]

The audit’s central recommendation is to make all dispatch paths obey the same mathematical obligations. F01 exposes an unbounded representation conversion; F02 exposes a wrapper-versus-primitive branch inconsistency; F03 exposes duplicated native-tail policies; F04 exposes inconsistent coefficient-domain enforcement; and F05 exposes a fixable mismatch between mathematical commensurability and the chosen encoding. These are more actionable than a broad assertion that the package needs “more features.”

The supplied patches are proposals, not a newly certified release. Native execution remains the next acceptance step. The repository’s own habit of publishing scoped test records is a good foundation: extend it to a complete release gate, preserve the distinction between formal and analytic evidence, and publish reproducible resource behavior. That would make the package a considerably stronger research tool without abandoning its current architecture or its cooperation with Wolfram Language.

A Reproducing this audit and using the companion artifacts

A.1 Archive layout

Artifact	Purpose and status
<code>article/asymptotic-audit.tex</code> and section files	Complete editable article source.
<code>article/asymptotic-audit.pdf</code>	Compiled article.
<code>code/independent_checks.py</code>	Fifteen executed mathematical and finite-model checks.
<code>code/emit_patch_proposals.py</code>	Fail-closed emitter of review patches and changed source copies; never edits the supplied checkout.
<code>code/test_patch_proposals.py</code>	Five executed text-transformation fixture tests; not Wolfram semantic validation.
<code>code/AuditRegressions.wlt</code>	Nine desired-behavior and control tests; native execution not obtained.
<code>code/RunAudit.wls</code>	Loads a local checkout, runs those tests, records native metadata, and invokes the tail probe.
<code>code/ProbeNativeTail.wls</code>	Records the actual native elliptic series shapes, old private fallback, and public result separately.
<code>code/CompareBuiltin.wls</code>	Twelve bounded native/package comparison calls; no results claimed here.
<code>results/</code>	Actual independent-test logs and machine-readable mathematical results.
<code>evidence/</code>	Snapshot, findings ledger, coverage, native-execution limitations, and source metadata.
<code>README.md</code>	Commands, dependencies, limitations, and patch-integration instructions.

The archive does not contain a full repository clone, a Wolfram kernel, or a compiled native test report. Repository access succeeded through source-reading tools, but the environment did not provide a mounted complete checkout for package execution. The patch emitter therefore validates a checkout supplied by the reader and produces proposals only when the expected source blobs match.

A.2 Actual independent checks

From the archive root, with Python, SymPy, and mpmath installed:

```
python code/independent_checks.py
python -m unittest discover -s code -p test_patch_proposals.py -v
```

The saved run used Python 3.13.5, SymPy 1.14.0, and mpmath 1.3.0. It passed all 15 independent checks and all five transformation checks. The mathematical checks cover allocation counts without allocation, rational rate normalization, coefficient formulas, resonance, an independent inverse polynomial, the elliptic logarithmic tail, a rational residual enclosure example, and the failure of differentiation of an arbitrary magnitude bound.

Neither test count should be presented as a count of passing repository tests. Several checks establish the reference mathematics for a proposed regression; others validate artifact-generation logic. They do not execute Wolfram evaluation rules, package upvalues, native special-function dispatch, formatting, or the complete repository suite.

A.3 Patch proposal generation

The emitter recognizes these exact normalized Git blob identities:

Canonical file	Git blob SHA-1
Core AsymptoticInverse.wl	ea9eaf4a11130e922ac4fb3faae37fd8e8d29643
FlatSectors.wl	293450f13e3ede4b490680f9ac0dea2f39fa2183

These are Git object identities, not raw-file SHA-256 digests. The emitter uses LF-normalized UTF-8 text to reconstruct the blob hash, so a checkout with ordinary line-ending conversion can still be recognized. A mismatch or missing anchor causes failure before output is written.

Use a checkout of the pinned commit and a new output directory outside it:

```
python code/emit_patch_proposals.py /path/to/Asymptotic \
/path/to/new-review-proposals
```

For the optional narrow F03 mitigation:

```
python code/emit_patch_proposals.py /path/to/Asymptotic \
/path/to/new-review-proposals \
--conservative-native-tails
```

The emitter writes changed source copies, a unified patch, and a manifest. Default proposals address F01, F02, and F05. The optional flag also changes the identified empty-native-tail branch. It does not implement F04, a typed result schema, lazy properties, or the full shared native importer.

The transformation tests exercise excerpts and anchor checks. The emitter was not run end to end against a complete local checkout during this audit, and the emitted Wolfram source was not executed. Review the patch, apply it only in a separate development branch, run the proposed regressions and the full native suite, then regenerate the standalone distribution using the repository's builder. Do not edit only the generated root file.

A.4 Native regression commands

On a machine with a compatible Wolfram kernel and a local checkout, set `ASYMPTOTIC_AUDIT_ROOT`. For example, in a POSIX shell:

```
export ASYMPTOTIC_AUDIT_ROOT=/path/to/Asymptotic
wolframscript -file code/RunAudit.wls
wolframscript -file code/CompareBuiltins.wls
```

In PowerShell, set the environment variable with

```
$env:ASYMPTOTIC_AUDIT_ROOT = "C:\path\to\Asymptotic"
```

The comparison and probe output files are written to the current working directory. Run in a dedicated results directory or preserve previous results before repeating a run.

The nine regression tests are *desired behavior* tests mixed with controls. Some are expected to fail on the pinned baseline. The F04 test remains a requested policy change even after applying the default patch proposals. A nonzero exit status therefore does not automatically mean the emitter corrupted the package; inspect test IDs and compare with the intended change set.

After modifying canonical kernel sources, the repository documents the following packaging checks:


```
python validation/build_standalone.py
python validation/build_standalone.py --check
python -m unittest discover -s validation \
    -p test_standalone.py -v
```

These are packaging checks, not replacements for native mathematical tests.[16, 18]

A.5 Compiling the article

From the `article` directory:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error \
    asymptotic-audit.tex
```

The source uses standard LaTeX packages and an embedded conventional bibliography; no external bibliography processor or proprietary fonts are required. All section files must remain beside the master source.

B Regression and acceptance matrix

Item	Desired invariant	Supplied evidence or next acceptance step
F01	A wide rational native grid cannot block a sparse result.	Exact allocation-count tests passed; bounded native desired-behavior test supplied. Measure preallocation/peak behavior, not just final list length.
F01	The native remainder order does not require trailing zero padding.	Emitter change and transformation tests supplied. Output-length control alone may already pass the baseline after native canonicalization.
F02	Pure-remainder real radicals are rejected through direct and recursive routes.	Source trace and nonreal mathematical example; two native desired/control tests supplied.
F03	An invisible logarithmic coefficient cannot be assigned degree zero without evidence.	Elliptic derivation and high-precision checks passed; private/public native-shape probe supplied.
F04	A real result cannot accept a provably nonreal constant coefficient silently.	Desired-behavior test supplied; broader coefficient policy is not patched.
F05	Positive rational rate ratios determine a fundamental rate, subject to degree limits.	Exact rational normalization tests passed; native sector example supplied.
Control	Complete logarithmic and irrational inverse blocks are preserved.	Independent coefficient checks and two native control tests supplied.
Control	Product remainder degree includes discarded boundary logarithms.	Native control test supplied; existing repository acceptance test corroborates intended behavior.[15]
Process	Release claims match the test selection and source revision.	Manifest and execution-status files included; native package result remains explicitly unverified.

C Coverage and source-navigation guide

Repository references below are immutable. Function names, not the line numbers of generated standalone output, are the most stable locators for the findings. The standalone file embeds sources and has different line positions. The cited commit is essential: later revisions may already change the relevant implementation.

Source group	Inspection focus and coverage boundary
Core kernel	Deep review of exact weights, sparse arithmetic, precision triples, ordinary forward fallback, endpoint normalization, Lagrange coefficients, inverse construction, native conversion, and public object behavior.
SeriesOperations and SeriesArithmetic	Deep review of representation extraction, arithmetic fallback, power/log/exp, observable recursion, composition, derivative admission, held normalization, and result recipes.
InverseFunctionExpressions	Branch/condition preprocessing, request-local caches, native inverse syntax recovery, dispatch, and provenance. Other branch/syntax modules were not exhaustively audited.
InverseCertificates	Rational enclosure primitives and main verification flow, including derivative separation, source-domain checks, existence, refinement, and stated certificate scope.
NativeSpecialFunctions and ParameterizedSpecialFunctions	Structured native-series import, absolute error transport, logarithmic tail policy, fixed-parameter/Frobenius preprocessing, and fallback budgets. Not every supported special function was independently checked.
GammaInverse	Family model restrictions, retained Lambert cores, coefficient recurrence, reciprocal-logarithmic remainder form, and option dispatch. Barnes-related interfaces and documentation reviewed; not every Barnes theorem or operation was re-proved.
FlatSectors	Exact monomial core, phase/rate admission, marker construction, finite degree operations, and sector-tail contract.
Guide, article, validation, tests	Current guide sections and feature map; article entry/organization and stated scope; current validation records; representative independent acceptance tests. Historical archives and all test files were not read exhaustively.
Distribution and metadata	Paclet declaration, root license, source header, root tree metadata, generated-distribution documentation, and the standalone freshness workflow. No full local clone or release build was obtained.

D An exact enclosure example independent of the package

To illustrate the distinction between a small residual and a certified interval, consider $f(x) = x + x^2$ and target $y = 1/100$. Choose

$$I = [9/1000, 11/1000], \quad c = 99/10000.$$

The derivative is $1 + 2x$, so it is at least $509/500$ throughout I . The exact residual is

$$f(c) - y = -\frac{199}{100000000}.$$

Hence the residual radius is

$$r = \frac{199}{101800000}.$$

The interval $[c - r, c + r]$ is contained in I . Since the derivative is positive, moving right by r increases f by at least the residual magnitude; continuity and monotonicity establish a unique root and the enclosure

$$x_* \in \left[\frac{1007621}{101800000}, \frac{1008019}{101800000} \right].$$

The included rational-arithmetic test verifies these relations. This is an independent illustrative calculation; it is not reported as an output of `InverseCertificate`. Its purpose is to make the certificate's existence and derivative obligations concrete.

References

- [1] V. Reshetnikov and contributors, *Pinned commit and change description*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/commit/75de8756175911cd8830704fd1a3406c1022f018>.
- [2] V. Reshetnikov and contributors, *Pinned repository tree and distribution metadata*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/tree/75de8756175911cd8830704fd1a3406c1022f018>.
- [3] V. Reshetnikov and contributors, *Core implementation: AsymptoticInverse.wl*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/AsymptoticInverse/Kernel/AsymptoticInverse.wl>.
- [4] V. Reshetnikov and contributors, *Explicit series operations: SeriesOperations.wl*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/AsymptoticInverse/Kernel/SeriesOperations.wl>.
- [5] V. Reshetnikov and contributors, *Ordinary arithmetic and held normalization: SeriesArithmetic.wl*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/AsymptoticInverse/Kernel/SeriesArithmetic.wl>.
- [6] V. Reshetnikov and contributors, *Applied inverse functions: InverseFunctionExpressions.wl*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/AsymptoticInverse/Kernel/InverseFunctionExpressions.wl>.
- [7] V. Reshetnikov and contributors, *Exact rational certificates: InverseCertificates.wl*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/AsymptoticInverse/Kernel/InverseCertificates.wl>.
- [8] V. Reshetnikov and contributors, *Structured native import: NativeSpecialFunctions.wl*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/AsymptoticInverse/Kernel/NativeSpecialFunctions.wl>.
- [9] V. Reshetnikov and contributors, *Fixed-parameter special functions: ParameterizedSpecialFunctions.wl*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/AsymptoticInverse/Kernel/ParameterizedSpecialFunctions.wl>.
- [10] V. Reshetnikov and contributors, *Gamma and Barnes inverse construction: GammaInverse.wl*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/AsymptoticInverse/Kernel/GammaInverse.wl>.
- [11] V. Reshetnikov and contributors, *Finite flat sectors: FlatSectors.wl*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/AsymptoticInverse/Kernel/FlatSectors.wl>.
- [12] V. Reshetnikov and contributors, *AsymptoticInverse User Guide*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/AsymptoticInverse/Documentation/UserGuide.md>.
- [13] V. Reshetnikov and contributors, *Real asymptotic expansions and inverse functions: article entry source*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/article/asymptotic-inverse.tex>.

- [14] V. Reshetnikov and contributors, *Review and validation record*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/validation/README.md>.
- [15] V. Reshetnikov and contributors, *AcceptanceCoordinatesCalculus.wlt: independent formulas and checks*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/AsymptoticInverse/Tests/AcceptanceCoordinatesCalculus.wlt>.
- [16] V. Reshetnikov and contributors, *Development, provenance, and standalone-distribution instructions*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/docs/development/README.md>.
- [17] V. Reshetnikov and contributors, *PacletInfo.wl: version and package metadata*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/AsymptoticInverse/PacletInfo.wl>.
- [18] V. Reshetnikov and contributors, *Standalone package freshness workflow*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/.github/workflows/standalone.yml>.
- [19] V. Reshetnikov and contributors, *Root LICENSE: MIT No Attribution License*, Asymptotic repository, pinned revision 75de875. <https://github.com/VladimirReshetnikov/Asymptotic/blob/75de8756175911cd8830704fd1a3406c1022f018/LICENSE>.
- [20] Wolfram Research, *Series*, Wolfram Language documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/Series.html>.
- [21] Wolfram Research, *SeriesData*, Wolfram Language documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [22] Wolfram Research, *InverseSeries*, Wolfram Language documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [23] Wolfram Research, *Asymptotic*, Wolfram Language documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [24] Wolfram Research, *AsymptoticSolve*, Wolfram Language documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [25] Wolfram Research, *AsymptoticIntegrate*, Wolfram Language documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/AsymptoticIntegrate.html>.
- [26] Wolfram Research, *AsymptoticSum*, Wolfram Language documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/AsymptoticSum.html>.
- [27] Wolfram Research, *AsymptoticDSolveValue*, Wolfram Language documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/AsymptoticDSolveValue.html>.
- [28] Wolfram Research, *AsymptoticRSolveValue*, Wolfram Language documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/AsymptoticRSolveValue.html>.
- [29] Wolfram Research, *FunctionDomain*, Wolfram Language documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/FunctionDomain.html>.
- [30] Wolfram Research, *EllipticK*, Wolfram Language documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/EllipticK.html>.
- [31] Wolfram Research, *TestReportObject*, Wolfram Language documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/TestReportObject.html>.
- [32] NIST Digital Library of Mathematical Functions, *§19.12: Asymptotic Approximations*, especially equations 19.12.1 and 19.12.3. Accessed September 9, 2026. <https://dlmf.nist.gov/19.12>.